Licenciatura en Tecnología Digital
TD4: Redes de Computadoras

Trabajo Práctico

Segundo semestre 2026

Introducción

En este trabajo práctico se construirá, sobre una red overlay inspirada en el subte de la Ciudad de Buenos Aires, el sistema que cobra el pasaje en cada molinete. El objetivo es recorrer las capas del modelo TCP/IP desde adentro: implementar el ruteo con el algoritmo de Dijkstra (red), un transporte confiable propio sobre UDP (transporte) y los protocolos con los que se comunican los molinetes, las cabeceras y el banco (aplicación), para luego analizar con Wireshark cómo viaja cada cobro y qué pasa cuando la red falla.

Contexto

Cada día cientos de miles de personas apoyan la tarjeta en un molinete del subte, esperan la confirmación, pasan el molinete y viajan. Detrás de esto, hay un sistema distribuido: el molinete genera una transacción, esa transacción tiene que llegar a un servidor que la autorice, el servidor consulta el saldo y la respuesta tiene que volver al pasajero. Al haber plata en juego, no alcanza con que el mensaje llegue sino que se debe confirmar que el cobro se efectuó correctamente y exactamente una vez.

En este trabajo tendrán que modelar ese sistema con cuatro tipos de procesos que corren en sus computadoras y se comunican por sockets:

- **Red de estaciones**: las estaciones del subte conforman una topología lógica representada mediante un grafo. Cada conexión entre estaciones tiene un costo asociado. A partir de esta topología, se deberán calcular mediante el algoritmo de Dijkstra los caminos de menor costo desde la estación de origen hacia las cabeceras disponibles.
- **Molinetes**: clientes asociados a una estación que generan transacciones de cobro, incluyendo la tarjeta, el monto y la estación de origen. A partir de la topología, el molinete deberá determinar qué cabecera alcanzable posee el menor costo y registrar la ruta calculada.
- **Cabeceras**: estaciones terminales que alojan un servidor gateway. El gateway recibe los cobros enviados por los molinetes y los reenvía al banco.
- **Banco**: un servidor central que mantiene el saldo de cada tarjeta y aprueba o rechaza cada cobro.

El recorrido de un cobro es el siguiente: el molinete genera la transacción y utiliza la topología de la red para calcular, mediante Dijkstra, la ruta de menor costo hacia cada cabecera disponible. Luego selecciona la cabecera alcanzable de menor costo e incorpora a la transacción tanto la cabecera elegida como la ruta calculada. En esta implementación simplificada, las estaciones intermedias de

esa ruta no funcionan como procesos independientes ni reenvían físicamente cada datagrama: la ruta representa la decisión lógica de encaminamiento. Una vez seleccionada la cabecera, el molinete envía la transacción al gateway mediante el protocolo confiable desarrollado sobre UDP. El gateway se comunica con el banco mediante HTTP y devuelve al molinete el resultado de la operación, indicando si el cobro fue aprobado o rechazado y, cuando corresponda, el saldo restante.

La red overlay no es perfecta: las estaciones pueden descartar paquetes y los enlaces pueden cortarse en cualquier momento. Por eso, para la comunicación entre el molinete y la cabecera deberán implementar un protocolo de transporte confiable sobre UDP.

Todos los grupos trabajan sobre el mismo mapa, pero cada uno tiene distinto:

- costos de enlace;
- topología resultante a partir de la semilla asignada;
- parámetros de pérdida utilizados en la experimentación;
- saldos iniciales de las tarjetas;
- estación de origen de los cobros;
- enlace a modificar o eliminar en la experimentación.

Así, un mismo cobro desde Medrano puede seguir un camino distinto en cada grupo, o incluso terminar en otra cabecera.

Se proporcionará un código base que deberán completar y ampliar para cumplir con todas las consignas. El código base incluye el generador de la topología, el formato de los mensajes, la estructura del molinete, el gateway y un servidor de banco mínimo.

Consignas

Parte 1: Capa de red, ruteo con Dijkstra

En esta parte deberán trabajar con la topología lógica del subte y calcular las rutas de menor costo entre estaciones. A partir del grafo generado para cada grupo, deberán implementar el algoritmo de Dijkstra y utilizarlo para determinar cuál es la cabecera alcanzable de menor costo desde la estación donde se origina el cobro. La ruta calculada se incorporará a la transacción como información del recorrido lógico.

Funciones a completar:

- **dijkstra(graph, source)**: recibe el grafo de la red y una estación de origen. Debe calcular el costo mínimo desde ese origen hacia todas las estaciones alcanzables.
- **shortest_path(graph, source, target)**: calcula y reconstruye la ruta de menor costo entre una estación de origen y una estación de destino. Si no existe un camino posible, deberá devolver una lista vacía.

- **build_transaction(card_id, amount, station)**: valida los datos del cobro, calcula el camino hacia cada cabecera disponible y selecciona aquella de menor costo. Luego genera el identificador de la transacción e incorpora la estación, la ruta y la cabecera elegida.

El archivo **topology.py** ya proporciona la función **build_topology(seed)**, que genera una topología reproducible para cada grupo, asignando costos a los enlaces y eliminando una conexión.

Parte 2: Capa de transporte, entrega confiable sobre UDP

En esta parte deberán implementar un mecanismo sencillo de transporte confiable sobre UDP. Como UDP no garantiza la entrega de los datagramas, el molinete deberá enviar cada mensaje y esperar una confirmación. Si el ACK no llega dentro del tiempo establecido, deberá retransmitir el mensaje hasta alcanzar el máximo de intentos permitido.

Funciones a completar:

- **send_reliably(address, payload, seq, timeout, max_retries, sock):** envía una transacción mediante UDP dentro de un mensaje DATA, espera un ACK con el mismo número de secuencia y retransmite en caso de timeout. Si se alcanza el máximo de reintentos sin recibir una confirmación válida, deberá generar un error.
- **serve(host, port):** corresponde al gateway. Debe abrir un socket UDP, recibir los mensajes enviados por los molinetes y responder con un ACK que contenga el mismo número de secuencia y el resultado de la operación. Además, debe conservar las respuestas ya procesadas para poder contestar una retransmisión sin volver a enviar el cobro al banco.

Parte 3: Capa de aplicación, protocolo de cobros

En esta parte deberán completar la lógica propia del sistema de cobros. Las transacciones se representarán como mensajes JSON que contienen los datos necesarios para identificar la operación, la tarjeta, el monto y la ruta calculada. El gateway recibirá esas transacciones por UDP y se comunicará con el banco mediante HTTP para solicitar la aprobación o rechazo del cobro.

Funciones a completar:

- **decode_message(data):** recibe los bytes obtenidos desde el socket, los decodifica desde UTF-8 y transforma el contenido JSON en un diccionario de Python. Debe detectar mensajes con formato inválido.
- **validate_transaction(tx):** verifica que la transacción posea los campos requeridos y que sus valores sean válidos, por ejemplo que el monto sea numérico y mayor que cero.

- **charge_bank(tx):** ejecutada por el gateway, valida la transacción y realiza una solicitud HTTP POST al banco enviando el cobro en formato JSON. Luego devuelve la respuesta obtenida o informa un error de comunicación.
- **process_transaction(tx):** ejecutada por el banco, valida la operación, verifica la existencia de la tarjeta y su saldo, y aprueba o rechaza el cobro. También controla que un mismo transaction_id no sea procesado más de una vez.
- **do_POST():** implementa el endpoint HTTP /charge. Recibe la transacción enviada por el gateway, llama a process_transaction() y devuelve el resultado en formato JSON. También debe manejar solicitudes inválidas y rutas desconocidas.

Ejecución del código

Para ejecutar la solución, deberán abrir tres terminales y correr los componentes del sistema en el siguiente orden:

- Terminal 1: cd 01_alumnos && python bank.py
- Terminal 2: cd 01_alumnos && python gateway.py
- Terminal 3: cd 01_alumnos && python turnstile.py

Por ejemplo, para realizar una operación desde el molinete, pueden ejecutar:
python turnstile.py --card SUBE-001 --amount 100 --station Medrano

Experimentación

En esta etapa deberán analizar el comportamiento del sistema frente a distintas condiciones de red y fallas controladas. El objetivo no es únicamente verificar que la implementación funcione, sino relacionar lo observado con los mecanismos desarrollados en las capas de red, transporte y aplicación.

En cada experimento deberán:

1. Escribir sus hipótesis antes de correrlo de qué esperan que ocurra y por qué.
2. Ejecutarlo y capturar el tráfico con Wireshark.
3. Comparar y analizar lo esperado con lo observado, justificando toda afirmación con evidencia de las capturas.

Experimento 1: Pérdida de mensajes y retransmisiones

Utilizando el mecanismo de pérdida provisto, analicen por separado dos situaciones:

- pérdida del mensaje DATA enviado por el molinete;
- pérdida del ACK enviado por el gateway.

En ambos casos deberán identificar en Wireshark y en los registros del programa la secuencia:

envío → pérdida o ausencia de respuesta → timeout → retransmisión.

Respondan:

a. ¿Qué comportamiento observan en cada caso y qué retransmisiones se producen?
b. ¿Cuántas veces llega la transacción al gateway y cuántas veces al banco?
c. ¿Por qué una retransmisión no debería producir un doble cobro?
d. ¿Qué diferencia hay entre el número de secuencia y *transaction_id*?

Experimento 2: Efecto de la pérdida y del timeout

Realicen una serie de cobros utilizando distintas tasas de pérdida de datagramas. Como mínimo deberán probar: 0 %, 5 %, 10 %, 20 %, 30 %.

Para cada condición, realicen una cantidad suficiente de ejecuciones como para poder comparar los resultados. Registren al menos: porcentaje de cobros completados; tiempo medio de respuesta; cantidad media de retransmisiones; cantidad de operaciones que alcanzaron el máximo de reintentos.

Luego repitan parte de las pruebas utilizando **al menos tres valores diferentes de timeout**: uno bajo, uno intermedio y uno alto.

Analicen:

- ¿Cómo afecta la pérdida al tiempo de respuesta y a las retransmisiones?
- ¿Qué ocurre cuando el timeout es demasiado bajo?
- ¿Qué ocurre cuando es demasiado alto?
- ¿Qué configuración consideran más adecuada y por qué?

Experimento 3: Falla de un enlace y recálculo de ruta

Mientras hay cobros en curso, corten el enlace asignado al grupo.

- ¿Cuánto tarda la red en volver a calcular las rutas? Muéstrenlo en la captura.
- Comparen el camino de un cobro desde la estación de origen antes y después del corte. ¿Cambió la cabecera de destino?
- ¿Qué pasó con los cobros que estaban en tránsito en el momento del corte? ¿Se perdió o se duplicó alguno?

Experimento 4: Diseño propio. Diseñen un experimento propio que ponga a prueba algún aspecto del sistema que no haya sido cubierto en los experimentos anteriores. Para esto deberán:

- Plantear la pregunta qué buscan responder y explicar por qué es relevante para un sistema de cobros real.
- Describir el escenario y qué van a medir. Debe ser reproducible: cualquier otro grupo podría repetirlo con su descripción.
- Concluir: ¿qué aprendieron sobre su implementación? ¿Cambiarían alguna decisión de diseño a partir del resultado?

Algunas posibilidades son: tarjeta inexistente; saldo insuficiente; monto inválido; banco no disponible; destino sin cabecera alcanzable; distintas estaciones de origen; comparación entre rutas con diferente cantidad de saltos y diferente costo.

Informe

Deberán realizar un informe con **máximo 8 páginas** (sin contar la carátula ni el anexo) que documente las decisiones relevantes tomadas durante la implementación, los errores encontrados y, en caso de haberlos solucionado, el proceso seguido para hacerlo. Además, deberán registrar de forma detallada los puntos de análisis y experimentación solicitados en la consigna.

El informe deberá tener como mínimo los siguientes apartados:

1. Introducción
2. Arquitectura del sistema
3. Decisiones de implementación (red, transporte y aplicación)
4. Experimentación: para cada experimento, hipótesis, metodología, resultados y discusión
5. Problemas encontrados y cómo se resolvieron
6. Conclusiones
7. Anexo: capturas de pantalla de Wireshark referenciadas en el informe, numeradas e identificadas por experimento

Modalidad de entrega y evaluación

El trabajo debe realizarse en **grupos de 4 alumnos**. Deberán entregar **un solo archivo `.zip` con sus apellidos** como nombre del mismo que contenga:

- Los códigos fuentes (**`.py`**) debidamente comentados.
- El informe en formato PDF.
- Las capturas de Wireshark

La entrega debe realizarse a través del campus virtual, con fecha límite el 13 de noviembre a las 23:59 hs.

No se aceptarán trabajos entregados fuera de término.

El trabajo será evaluado de la siguiente manera: **Código x 0,4 + Informe x 0,6**. Donde el código debe correr sin problemas y realizar lo pedido, y el informe debe mostrar lo trabajado, con cualquier problema que pueda haber surgido. Adicionalmente, se realizará una **defensa oral obligatoria de 15 minutos** por grupo, la fecha se confirmará en cada sección. Esta defensa tiene como objetivo constatar el trabajo realizado y, según el desempeño durante la misma, podrá impactar en la nota

final del trabajo práctico.

Documentación

Para realizar el trabajo práctico, van a necesitar investigar distintas herramientas de la biblioteca estándar de Python. El objetivo de esta sección es orientar la búsqueda de documentación, no proporcionar una implementación de las funciones solicitadas. Se recomienda consultar la documentación oficial y utilizarla para comprender cómo funcionan las herramientas antes de incorporarlas al código. No es necesario conocer de antemano todas las bibliotecas: parte del trabajo consiste en aprender a buscar, interpretar y aplicar documentación técnica.

1. Estructuras de datos y algoritmos

Biblioteca: heapq
Documentación oficial:

- https://docs.python.org/3/library/heapq.html
- https://docs.python.org/3/builtins/stdtypes.html#str.encode

2. Sockets y transporte

Biblioteca: socket
Documentación oficial: https://docs.python.org/3/library/socket.html

3. Formato y validación de mensajes

Biblioteca: json
Documentación oficial: https://docs.python.org/3/library/json.html

4. Cliente y servidor HTTP

Bibliotecas: urllib.request, urllib.error y http.server
Documentación oficial:

- Cliente HTTP (urllib.request): https://docs.python.org/3/library/urllib.request.html
- Manejo de errores (urllib.error): https://docs.python.org/3/library/urllib.error.html
- Servidor HTTP (http.server): https://docs.python.org/3/library/http.server.html

Recomendación general
La documentación oficial describe las herramientas disponibles, pero no necesariamente explica cómo resolver el problema específico del trabajo práctico. Cada grupo deberá identificar qué herramientas necesita, comprender su funcionamiento y justificar las decisiones tomadas al utilizarlas. El objetivo no es memorizar la documentación, sino desarrollar la capacidad de investigar bibliotecas, interpretar documentación técnica y aplicar lo aprendido a un problema de redes.